Inclusion

• Visão Geral Do Sistema

O nosso sistema trata-se de uma rede social que pretende facilitar a comunicação e disseminação de informação entre indivíduos.

De modo a disseminar informação os utilizadores do nosso sistema podem publicar diversos tipos de conteúdo. Os utilizadores podem optar por produzir um tipo de conteúdo mais curto, como histórias ou notas, ou por um tipo de conteúdo mais longo como posts ou incs.

Os utilizadores podem também definir o estado da sua conta como privada, tornando o seu conteúdo disponível apenas para amigos. Deste modo, verifica-se também a possibilidade de os utilizadores enviarem pedidos de amizade, podendo aceitar ou recusá-los, de acordo com a vontade do utilizador ao qual foi efetuado o pedido, tornando a amizade efetiva ou não.

Os utilizadores podem comunicar entre si através de mensagens diretas em tempo real.

O sistema dispõe de um jogo diário baseado no Wordle, que consiste em adivinhar a palavra diária proposta pelo sistema diariamente.

O sistema dispõe de notificações em tempo real de modo a alertar os utilizadores aquando de acontecimentos que envolvam a sua conta.

Como motivação para o desenvolvimento desta rede social surgiu a vontade de alterar o paradigma das redes sociais infestadas por inteligência artificial, bots, fake news, ódio e discriminação, promovendo no seu lugar a inclusão, solidariedade e confraternidade.

- # Diagrama de Arquitetura de Alto Nível

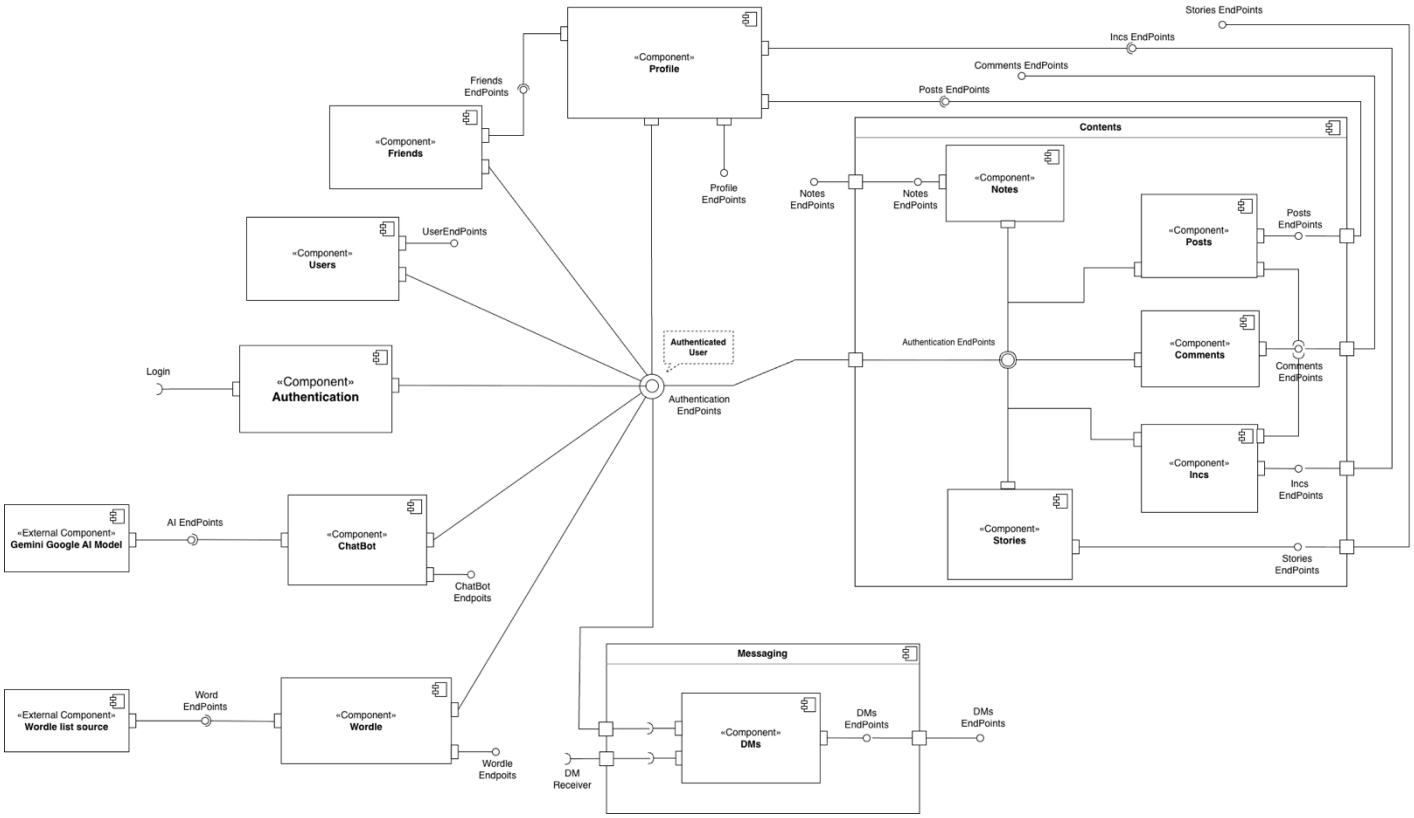


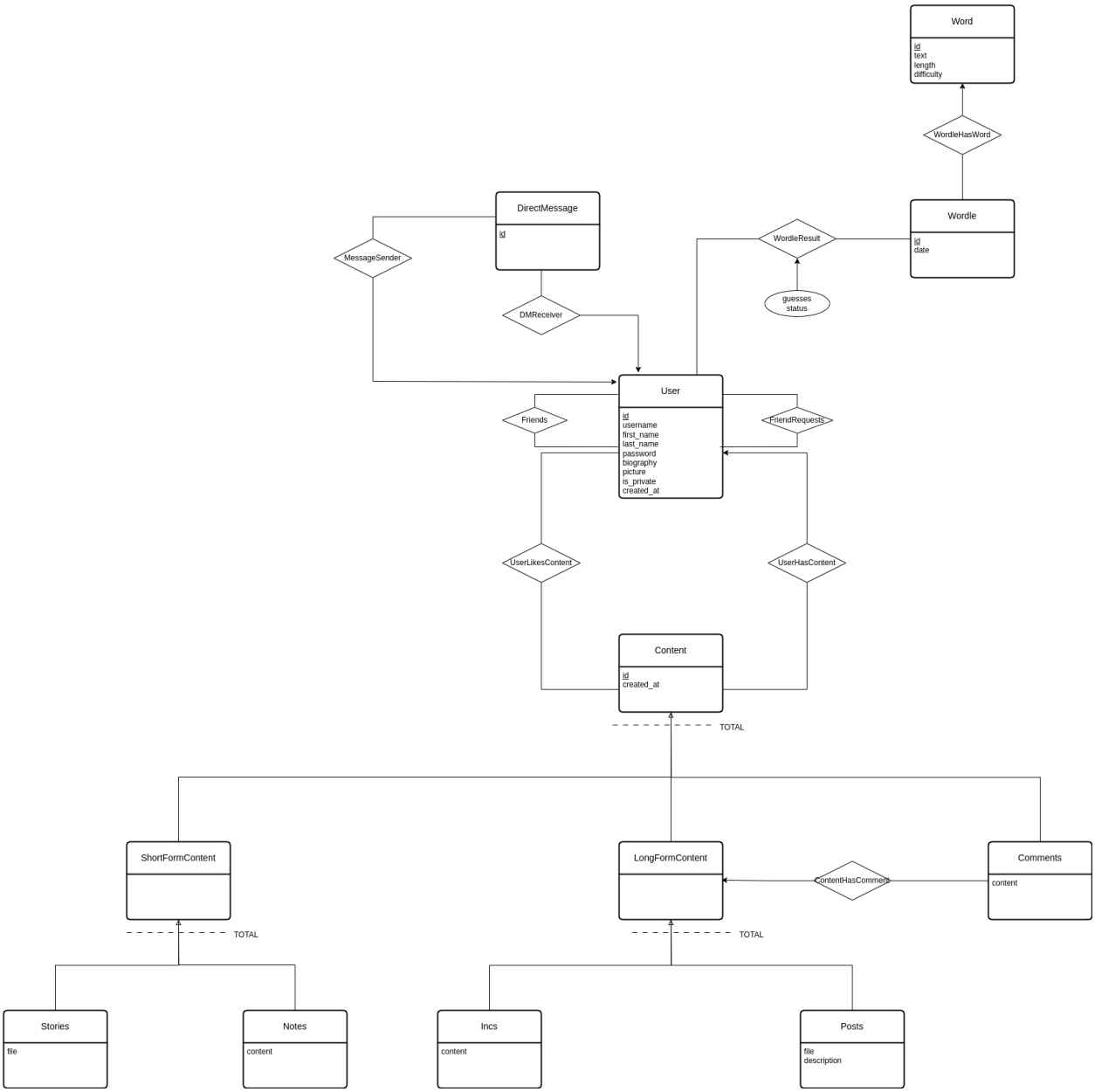
Diagrama de componentes (*Pode também ver o modelo com mais detalhe ao carregar neste link)*

Nota: O componente Messaging foi criado a pensar numa futura implementação de mensagens com grupos.

- # Modelo de Dados

Apresenta-se o modelo de dados da nossa base de dados em modelo entidade relação e relacional.

- **Entidade Relação** *(Pode também ver o modelo com mais detalhe ao carregar neste link)*



▪ Relacional

User (id, username, first_name, last_name, password, biography, picture, is_private, created_at)

NN: id
UK: username
NN: username, first_name, last_name, password, picture, is_private, created_at

---

Friends (*user_1, user_2*)

FK:
- user_1 → User
- user_2 → User

RI:
- user_1.id < user_2.id
- user_1.id != user_2.id

FriendRequest (*from_user_id, to_user_id,* status)

FK:
- from_user_id → User
- to_user_id → User

NN: status
RI: status só pode ser Accepted, Rejected ou Pending

DirectMessage (id, *sender, receiver*)

PK: **id**
FK:
- sender → User
- receiver → User

NN: receiver, sender

Content (id, created_at)

PK: id

NN: created_at

ShortFormContent (*id*)

FK: id → Content

LongFormContent (*id*)

FK: id → Content

Comments (id, content, *long_form_content_id*)

PK: id

FK: long_form_content_id → LongFormContent

NN: content, long_form_content_id

Stories (*id*, file)

FK: id → ShortFormContent

NN: file

Notes (*id*, content)

FK: id → ShortFormContent

NN: content

Incs (*id*, content)

FK: id → LongFormContent

NN: content

Posts (*id*, file, description)

FK: id: → LongFormContent

NN: file, description

RI: O file só pode ser um vídeo ou imagem.

Word (id, text, length, difficulty)

PK: id

NN: text, length, difficulty

RI: Difficulty só pode ser easy, medium ou hard

Wordle (*id*, date, word)

FK: id → Word

NN: date, word

WordleResult (*user_id*, *wordle_id*, guesses, status)

FK:

- user_id → User
- wordle_id → Wordle

NN: guesses, status

RI:

- guesses >= 0
- Status só pode ser success ou failure.

- ## Justificação das decisões técnicas e arquiteturais

- **Common**

  - ### Docker

A utilização do Docker permitiu orquestrar toda a infraestrutura do projeto, abrangendo o backend, o frontend e os serviços PostgreSQL e Redis. Esta solução foi escolhida pela sua eficácia na gestão de múltiplos contentores e pela familiaridade da equipa, garantindo que todos os componentes correm de forma isolada, consistente e simplificada em qualquer ambiente de execução.

  - ### Continuous Integration & Continuous Delivery

Implementaram-se fluxos de CI/CD via GitHub Actions para o frontend e backend, garantindo a estabilidade e integridade do código em cada pull request ou push para a branch principal. Adicionalmente, estes workflows automatizam a compilação da aplicação Tauri para múltiplas plataformas, assegurando a geração eficiente de builds para Windows, Linux, macOS e Android em cada ciclo de entrega.

- **Backend**

  - ### Python

Decidiu-se utilizar Python dada a familiaridade de toda a equipa com a linguagem, devido a inúmeras cadeiras que têm esta linguagem de programação como requisito e também pelo facto dos membros da equipa terem alguns projetos externos ao contexto académico nos quais utilizaram esta linguagem de programação como principal ferramenta.

  - ### Django Rest Framework

Decidiu-se utilizar Django Rest Framework no desenvolvimento do backend devido ao ecossistema robusto e ao toolkit poderoso e flexível para construir REST APIs,

oferecendo features como autenticação, serialização e uma ORM que é possível integrar com a base de dados com facilidade. O facto ser altamente customizável e permitir o desenvolvimento ágil de uma REST API também vai ao encontro da metodologia utilizada ao longo do desenvolvimento do projeto (SCRUM).

Em termos de autenticação utilizou-se JSON Web Tokens (JWT) como método de autenticação na nossa REST API.

Utilizou-se cronjobs de modo a gerar o daily wordle e de modo a garantir o update das streaks do wordle de cada utilizador.

Promoveu-se a segurança da API desenvolvida, validando todos os dados recebidos, incluindo tamanho e extensão no caso de ficheiros.

Desenvolveu-se testes unitários automáticos tomando partido do módulo fornecido para tal efeito pelo Django Rest Framework.

- **Swagger**

Decidiu-se utilizar Swagger de modo a documentar toda a REST API desenvolvida.

Tem como principal objetivo dar a conhecer à equipa de frontend como a REST API está desenvolvida e como comunicar com a mesma.

- **PostgreSQL**

Decidiu-se utilizar PostgreSQL como base de dados pois devido às suas features poderosas como reliability e flexibilidade. Para além disso é também open-source, o que a torna muito apelativa para nós dado o baixo orçamento do qual a equipa dispõe.

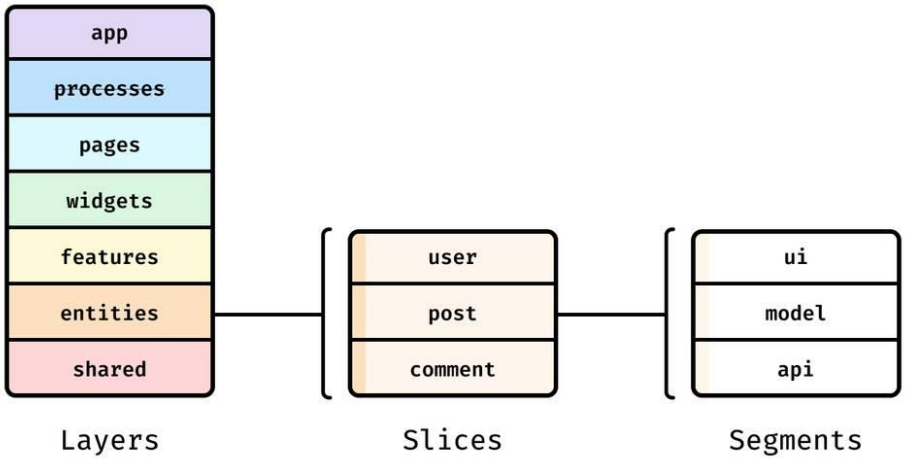
- **Redis**

Utilizou-se Redis como message broker para as mensagens diretas em tempo real entre utilizadores.

- **Frontend**

  - **Feature-Sliced Design**

A adoção da arquitetura Feature-Sliced Design (FSD) neste projeto foi motivada pela necessidade de garantir uma base de código altamente modular, escalável e de fácil manutenção. Esta metodologia proporciona uma separação clara de responsabilidades ao organizar a aplicação em camadas lógicas focadas nas funcionalidades e no domínio do negócio, em detrimento de uma estruturação puramente técnica.



- **ReactJS**

O ReactJS foi escolhido por ser a biblioteca mais popular para criar Single Page Applications (SPA). A sua principal vantagem é o vasto ecossistema de bibliotecas e uma comunidade gigantesca, o que garante excelente suporte técnico, evolução contínua e facilidade de desenvolvimento.

- **Vite**

A adoção do Vite justifica-se por se ter tornado o padrão moderno no desenvolvimento frontend. A ferramenta foi escolhida por oferecer um ambiente extremamente rápido (com HMR quase instantâneo) e por possuir um excelente ecossistema de plugins que otimizam o fluxo de trabalho.

- **TypeScript**

A escolha do TypeScript justifica-se por ser o atual padrão da indústria para o desenvolvimento frontend moderno. A sua principal vantagem é a introdução de tipagem estática, que garante uma maior segurança ao código e permite detetar erros logo na fase de desenvolvimento. Esta abordagem resulta numa redução drástica de falhas em produção e melhora significativamente a produtividade da equipa através de um excelente suporte de ferramentas de edição e refatoração segura.

- **ESLint**

O ESLint foi usado para garantir a qualidade do código e prevenir más práticas. A ferramenta analisa o código de forma estática para detetar e corrigir erros antes da execução, mantendo o projeto alinhado com os padrões da indústria e facilitando a sua manutenção.

- **Prettier**

A utilização do Prettier serve para assegurar uma formatação de código totalmente consistente. Ao formatar o código de forma automática, elimina debates sobre o estilo, garante uma aparência uniforme e aumenta a legibilidade para toda a equipa.

- **Axios**

O Axios foi selecionado para gerir a comunicação com as APIs devido à sua robustez e funcionalidades avançadas, como os intercetores e a transformação automática de dados. Esta biblioteca simplifica o tratamento de erros e a configuração de pedidos HTTP, garantindo um fluxo de dados mais consistente e seguro em comparação com as soluções nativas.

- **Shadcn/ui**

O shadcn/ui foi adotado por oferecer um conjunto de componentes acessíveis e reutilizáveis que garantem uma interface moderna e consistente. A sua principal vantagem reside na flexibilidade, permitindo o controlo total sobre o código-fonte de

cada elemento, o que facilita a personalização profunda e acelera significativamente o processo de desenvolvimento sem comprometer a qualidade visual.

- **TailwindCSS**

A escolha do TailwindCSS deve-se à sua metodologia "utility-first", que permite acelerar drasticamente o desenvolvimento da interface e garantir uma consistência visual rigorosa. Esta ferramenta elimina a necessidade de escrever CSS customizado complexo, facilitando a criação de designs responsivos e altamente escaláveis diretamente no código.

- **I18next**

O i18next foi selecionado como a solução de internacionalização (i18n) devido à sua robustez e flexibilidade. Esta biblioteca é o padrão da indústria, permitindo gerir traduções de forma escalável, com suporte para carregamento dinâmico de idiomas e deteção automática de preferências do utilizador, o que assegura uma experiência localizada de alta qualidade.

- **Tauri**

A escolha do Tauri baseou-se na possibilidade de compilar a aplicação tanto para desktop como para mobile a partir do mesmo código frontend. É uma solução significativamente mais leve que o Electron, pois utiliza o motor nativo do sistema operativo, permitindo uma distribuição rápida sem necessidade de adaptar ou reescrever a interface.

- **TanStack**:

  - **Query**: foi adotado para gerir o estado do servidor de forma eficiente. A biblioteca automatiza o cache, a sincronização e a atualização de dados em segundo plano, eliminando a necessidade de lógica complexa para pedidos manuais e melhorando a performance da aplicação.
  - **Router**: a escolha deve-se ao seu sistema de rotas totalmente tipado. Isto garante que os erros de navegação sejam detetados durante o desenvolvimento,

além de oferecer uma gestão de estados de URL e carregamento de dados muito mais robusta que as soluções tradicionais.

- **Forms**: foi selecionado por ser uma solução leve e agnóstica para a gestão de formulários. A sua principal vantagem é a tipagem estrita de todos os campos e validações, o que evita erros de submissão e garante uma excelente performance, mesmo em formulários complexos.

- ## Zod

Foi escolhido para garantir a validação rigorosa de esquemas e dados em tempo de execução. A sua integração perfeita com o TypeScript permite definir contratos de dados claros para APIs e formulários, assegurando que a aplicação lida apenas com informações válidas e seguras.

- ## Zustand

A adoção do Zustand justifica-se pela sua simplicidade e leveza na gestão de estado global. É uma alternativa mais eficiente e com menos boilerplate do que o Redux, permitindo uma gestão de estados centralizada, rápida e extremamente fácil de manter.